

Snapdeal Customer Survey Analysis

A simple, no-jargon walkthrough of what I did in this project — task by task, with screenshots.

What this project is about: Snapdeal (well, a survey dataset styled like an e-commerce customer survey) collected responses from 800 people about how they shop online — how often they buy stuff, why they abandon their cart, whether reviews and recommendations actually influence them, and how satisfied they are overall. My job was to clean this messy survey data, dig into it, group customers into meaningful buckets, and present the findings with charts. The whole thing was broken into 5 tasks by the internship — I'll explain each one like I'm explaining it to a classmate, not like a textbook.

Tools used: Python, Pandas (for cleaning + analysis), Matplotlib (for the charts), and Scikit-learn (for the K-Means clustering part in Task 3). Everything was done in a Jupyter Notebook.

Task 1: Data Cleaning and Preparation

Basically, before you can analyze any data, you have to make sure it's not messy. Real survey data is almost never clean — people leave fields blank, column names get typos, values are written differently by different people ('Yes' vs 'yes' vs 'YES'), and so on. So this whole task was just cleanup work.

Step 1 - Checking for duplicate rows and fixing column names

First thing I did was check if any survey response got submitted twice by mistake, using `data.duplicated().sum()`. Got a clean 0, so no duplicates — nice. Then I noticed two columns in the dataset were literally both named `Personalized_Recommendation_Frequency` (a copy-paste mistake from whoever made the survey form/export). Pandas doesn't like having two columns with the exact same name because you can't tell them apart when you call them, so I renamed them to `Recommendation_Frequency_Text` and `Recommendation_Frequency_Score` based on what kind of values they actually held.

```
# 1 . Duplicate

print(data.duplicated().sum())

0

# 2. Rename Duplicate

print(data.columns.str.strip())

... Index(['Timestamp', 'age', 'Gender', 'Purchase_Frequency',
        'Purchase_Categories', 'Personalized_Recommendation_Frequency',
        'Browsing_Frequency', 'Product_Search_Method',
        'Search_Result_Exploration', 'Customer_Reviews_Importance',
        'Add_to_Cart_Browsing', 'Cart_Completion_Frequency',
        'Cart_Abandonment_Factors', 'Saveforlater_Frequency', 'Review_Left',
        'Review_Reliability', 'Review_Helpfulness',
        'Personalized_Recommendation_Frequency', 'Recommendation_Helpfulness',
        'Rating_Accuracy', 'Shopping_Satisfaction', 'Service_Appreciation',
        'Improvement_Areas', 'transaction'],
        dtype='object')

cols = data.columns.tolist()
cols[5] = 'Recommendation_Frequency_Text'
cols[17] = 'Recommendation_Frequency_Score'
data.columns = cols
```

Screenshot: checking duplicates and renaming the clashing column names

A quick note on why this matters: if you don't rename clashing columns, pandas will still let you run your code, but the moment you try to select that column by name, it gets confused about which one you mean and can throw errors or silently grab the wrong one. So catching this early saved me a lot of headache later when I actually needed those two columns separately in Task 3 and Task 4.

```
# Renamed Columns
print(data.columns.str.strip())

Index(['Timestamp', 'age', 'Gender', 'Purchase_Frequency',
       'Purchase_Categories', 'Recommendation_Frequency_Text',
       'Browsing_Frequency', 'Product_Search_Method',
       'Search_Result_Exploration', 'Customer_Reviews_Importance',
       'Add_to_Cart_Browsing', 'Cart_Completion_Frequency',
       'Cart_Abandonment_Factors', 'Saveforlater_Frequency', 'Review_Left',
       'Review_Reliability', 'Review_Helpfulness',
       'Recommendation_Frequency_Score', 'Recommendation_Helpfulness',
       'Rating_Accuracy', 'Shopping_Satisfaction', 'Service_Appreciation',
       'Improvement_Areas', 'transaction'],
      dtype='object')
```

Screenshot: confirming the columns were renamed properly

Step 2 - Hunting down missing values

Next I ran `data.isna().sum()` to see which columns had empty cells. Turned out only one column had a problem — `Product_Search_Method` had 149 blank responses (people just skipped that question). Rather than deleting those 149 rows and losing all their other answers, I filled the blanks with the label "Unknownn" so I don't lose useful data from the rest of their responses. After that, the missing value count came back all zeros.

```
# missing values

print(data.isna().sum())

Timestamp          0
age                0
Gender             0
Purchase_Frequency 0
Purchase_Categories 0
Recommendation_Frequency_Text 0
Browsing_Frequency 0
Product_Search_Method 149
Search_Result_Exploration 0
Customer_Reviews_Importance 0
Add_to_Cart_Browsing 0
Cart_Completion_Frequency 0
Cart_Abandonment_Factors 0
Saveforlater_Frequency 0
Review_Left        0
Review_Reliability 0
Review_Helpfulness 0
Recommendation_Frequency_Score 0
Recommendation_Helpfulness 0
Rating_Accuracy    0
Shopping_Satisfaction 0
Service_Appreciation 0
Improvement_Areas  0
transaction        0
dtype: int64
```

Screenshot: spotting the 149 missing values in Product_Search_Method

The screenshot below shows the fix in action — a single line using `fillna("Unknownn")`, followed by re-running the missing value check to prove it actually worked. I know "Unknownn" has a typo with the extra 'n' — I noticed it after running the cell but kept it since it doesn't affect the analysis, it's just a label. Small reminder that data cleaning code doesn't have to be perfect, it just has to work correctly on the data.

```
data['Product_Search_Method'] = data['Product_Search_Method'].fillna("Unknown")

print(data.isna().sum())
```

Timestamp	0
age	0
Gender	0
Purchase_Frequency	0
Purchase_Categories	0
Recommendation_Frequency_Text	0
Browsing_Frequency	0
Product_Search_Method	0
Search_Result_Exploration	0
Customer_Reviews_Importance	0
Add_to_Cart_Browsing	0
Cart_Completion_Frequency	0
Cart_Abandonment_Factors	0
Saveforlater_Frequency	0
Review_Left	0
Review_Reliability	0
Review_Helpfulness	0
Recommendation_Frequency_Score	0
Recommendation_Helpfulness	0
Rating_Accuracy	0
Shopping_Satisfaction	0
Service_Appreciation	0
Improvement_Areas	0
transaction	0
dtype: int64	

Screenshot: filling missing values and confirming all columns are now at zero missing

Step 3 - Standardizing text and saving the clean file

People type the same answer in different cases and with extra spaces, which messes up grouping later (pandas treats 'Male ' and 'Male' as two different values). So I looped through every text column, printed out the unique values to check them, and standardized the casing (title case) so everything is consistent. I also converted rating columns like Shopping_Satisfaction into proper numeric types so I could actually do math on them later.

This step took the longest simply because there are so many text columns in this survey — 20+ of them. I had to go column by column, print out every unique value, and check for anything weird: extra spaces, inconsistent capitalization (like 'yes' and 'Yes' showing up as two different answers), or category labels written in slightly different ways. Once everything was standardized, I did one final dtype check to make sure all the numeric columns (age, satisfaction score, rating accuracy, etc.) were actually stored as numbers and not as text — because if a rating column is secretly stored as text, `.mean()` and other math functions will just throw an error or give nonsense.

```
# --- 6. Confirm numeric dtypes ---
print(data.dtypes)
```

Timestamp	object
age	int64
Gender	object
Purchase_Frequency	object
Purchase_Categories	object
Recommendation_Frequency_Text	object
Browsing_Frequency	object
Product_Search_Method	object
Search_Result_Exploration	object
Customer_Reviews_Importance	int64
Add_to_Cart_Browsing	object
Cart_Completion_Frequency	object
Cart_Abandonment_Factors	object
Saveforlater_Frequency	object
Review_Left	object
Review_Reliability	object
Review_Helpfulness	object
Recommendation_Frequency_Score	int64
Recommendation_Helpfulness	object
Rating_Accuracy	int64
Shopping_Satisfaction	int64
Service_Appreciation	object
Improvement_Areas	object
transaction	int64
dtype:	object

Screenshot: confirming every column has the correct data type before moving on

At the very end, I ran a final check to confirm zero missing values, and saved everything into a fresh file called `snapdeal_cleaned.csv` — this is the file every other task actually works on.

```

# --- final check ---
print(data.isna().sum())      # should be near 0 row
print(data.columns.tolist())  # confirm renamed columns look right

data.to_csv('snapdeal_cleaned.csv', index=False)

Timestamp      0
age             0
Gender         0
Purchase_Frequency 0
Purchase_Categories 0
Recommendation_Frequency_Text 0
Browsing_Frequency 0
Product_Search_Method 0
Search_Result_Exploration 0
Customer_Review_Importance 0
Add_to_Cart_Browsing 0
Cart_Completion_Frequency 0
Cart_Abandonment_Factors 0
Save_for_Later_Frequency 0
Review_Left    0
Review_Reliability 0
Review_Helpfulness 0
Recommendation_Frequency_Score 0
Recommendation_Helpfulness 0
Rating_Accuracy 0
Shopping_Satisfaction 0
Service_Appreciation 0
Improvement_Areas 0
Transaction     0
dtype: object
['Timestamp', 'age', 'Gender', 'Purchase_Frequency', 'Purchase_Categories', 'Recommendation_Frequency_Text', 'Browsing_Frequency', 'Product_Search_Method', 'Search_Result_Exp

```

Screenshot: final missing-value check and saving snapdeal_cleaned.csv

In short: Task 1 was all prep work — no duplicates, no clashing column names, no blank cells, and consistent text formatting. Boring but super important, because if this step is wrong, every chart and number after this is wrong too.

Task 2: Descriptive Behavior Analysis

This task is basically just exploring the data to understand who's answering the survey and how they shop. No fancy models here, just counting, averaging, and a few charts to get a feel for the numbers.

Who's actually filling this survey out?

I used `data['age'].describe()` to get the average age, spread, min and max in one shot — average age came out to around 36. Then I checked the gender split using `value_counts()`, and also converted it to percentages so it's easier to read. It's a pretty even split across Male, Female, Others, and Prefer not to say — nobody dominates.

```
# task 2
#1. Demographics (age, gender distribution)
print(data['age'].describe())           # mean, median, spread, min/max
data['Gender'].value_counts()           # counts per gender
data['Gender'].value_counts(normalize=True) * 100  # as percentages

count      800.000000
mean       36.053750
std        19.310087
min         3.000000
25%        18.000000
50%        37.000000
75%        53.000000
max        67.000000
Name: age, dtype: float64

              proportion
Gender
Male          26.625
Female        25.125
Prefer Not To Say 24.250
Others         24.000

dtype: float64
```

Screenshot: age stats and gender distribution as percentages

How often do people actually shop?

Then I looked at `Purchase_Frequency` to see how regularly people are buying on Snapdeal — dailyish shoppers vs monthly shoppers vs the occasional buyer. The counts came out surprisingly balanced across all five frequency levels (from 'Multiple times a week' at 166 down to 'Less than once a month' at 152), which tells me this survey sample isn't skewed toward any one type of shopper — it's a good mix of heavy users and light users.



Screenshot: purchase frequency counts across the sample

How do people browse and why do they ditch their cart?

Next I checked how people search for products and, more interestingly, why they abandon items in their cart without buying. Turns out "Found a better price elsewhere" is the top reason (219 people said this), closely followed by high shipping costs and just changing their mind. This is genuinely useful info — it tells Snapdeal that price competitiveness matters more than anything else for closing a sale.

```
#3. Browsing methods & cart abandonment factors
data['Product_Search_Method'].value_counts()
data['Cart_Abandonment_Factors'].value_counts()
```

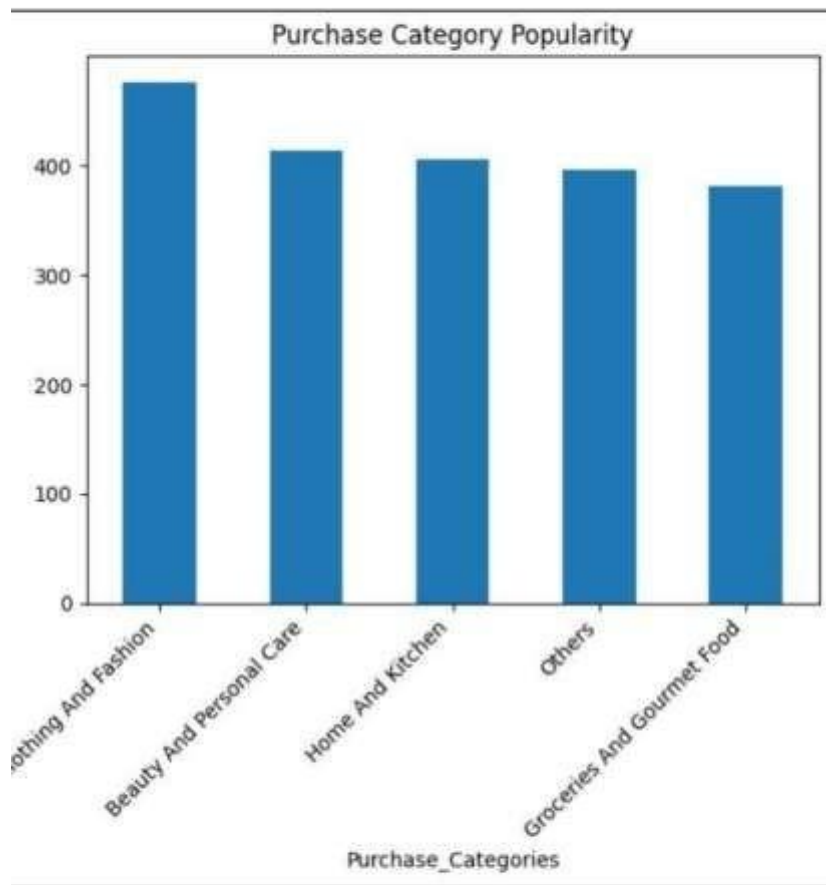
	count
Cart_Abandonment_Factors	
Found A Better Price Elsewhere	219
Others	196
High Shipping Costs	194
Changed My Mind Or No Longer Need The Item	191

dtype: int64

Screenshot: cart abandonment reasons, counted

Turning the numbers into charts

Numbers in a table are hard to read at a glance, so I plotted them. Made a pie chart for gender split, a histogram for age distribution, and a bar chart showing which product categories people buy the most (after splitting the multi-category answers apart, since one person could pick multiple categories). Clothing and Fashion came out on top as the most popular category.



Screenshot: bar chart of most popular purchase categories

In short: Task 2 is the "getting to know your data" step — average shopper is ~36 years old, genders are fairly balanced, price is the #1 reason people abandon carts, and Clothing & Fashion is the most-bought category.

Task 3: Customer Segmentation and Profiling

This is the interesting part. Instead of treating all 800 customers the same, I split them into groups (segments) based on how often they buy and how satisfied they are. This is exactly what real companies do — you don't market the same way to a loyal daily shopper and someone who's about to churn.

Rule-based segmentation (the simple way first)

I wrote a simple function that checks each customer's purchase frequency score and satisfaction score, and buckets them into one of three labels: **Frequent Buyer** (buys often AND is happy), **At-Risk** (satisfaction is low — these are the people Snapdeal could lose), or **Occasional Shopper** (everyone else, the middle ground). Applied this rule to every row using `data.apply()`. Out of 800 people, 352 are

Occasional, 320 are At-Risk, and only 128 are truly Frequent Buyers — which honestly is a bit concerning for the business since At-Risk is a huge chunk.

```
# Task 3
def assign_segment(row):
    if row['Purchase_Frequency_Score'] >= 4 and row['Shopping_Satisfaction'] >= 4:
        return 'Frequent Buyer'
    elif row['Shopping_Satisfaction'] <= 2:
        return 'At-Risk'
    else:
        return 'Occasional Shopper'

data['Segment'] = data.apply(assign_segment, axis=1)
print(data['Segment'].value_counts())
```

Segment	
Occasional Shopper	352
At-Risk	320
Frequent Buyer	128

Name: count, dtype: int64

Screenshot: the segment-assigning function and the resulting counts

Do these segments actually look different from each other?

Once the segments were created, I didn't just want counts — I wanted to know if these groups genuinely behave differently, or if the labels are basically meaningless. So I checked average age, gender split, and the actual purchase-frequency/satisfaction numbers per segment using `groupby()` and a `crosstab()`. Interestingly, age is almost identical across all three groups (around 36 for everyone), so age isn't what's driving the difference. But purchase frequency and satisfaction scores are, as expected, night and day between Frequent Buyers (avg satisfaction 4.56) and At-Risk customers (avg satisfaction just 1.51) — which confirms the segmentation logic is actually capturing something real, not just splitting data randomly.

```
#2. Demographic/behavioral differences across segments
print(data.groupby('Segment')['age'].mean())
print(pd.crosstab(data['Segment'], data['Gender']))
print(data.groupby('Segment')[['Purchase_Frequency_Score', 'Shopping_Satisfaction']].mean())
```

Segment	age
At-Risk	36.162500
Frequent Buyer	35.984375
Occasional Shopper	35.980114

Name: age, dtype: float64

Gender	Female	Male	Others	Prefer Not To Say
Segment				
At-Risk	84	74	84	78
Frequent Buyer	27	36	35	30
Occasional Shopper	90	103	73	86

	Purchase_Frequency_Score	Shopping_Satisfaction
Segment		
At-Risk	3.084375	1.512500
Frequent Buyer	4.484375	4.562500
Occasional Shopper	2.454545	3.758523

Screenshot: comparing age, gender, and average scores across the three segments

Letting the machine find groups on its own (K-Means)

My rule-based segments work fine, but they're based on my own if/else logic — basically my opinion of what makes a good segment. So I also ran **K-Means clustering**, which is an algorithm that looks at the numbers (purchase frequency + satisfaction, scaled to the same range) and figures out natural groupings on its own, without me telling it any rules. Before running it, I used the "Elbow Method" — basically trying cluster counts from 2 to 7 and plotting how the error drops each time — to decide that 3 clusters was the sweet spot (that's the bend/elbow in the graph). Then I ran K-Means with `n_clusters=3` and it split customers into 3 groups of roughly similar sizes (316, 250, and 234 customers each) — pretty close to what my rule-based segments came up with. Seeing two completely different approaches agree on roughly the same number of natural groups gave me more confidence that '3 customer types' isn't just something I made up, it's actually there in the data.

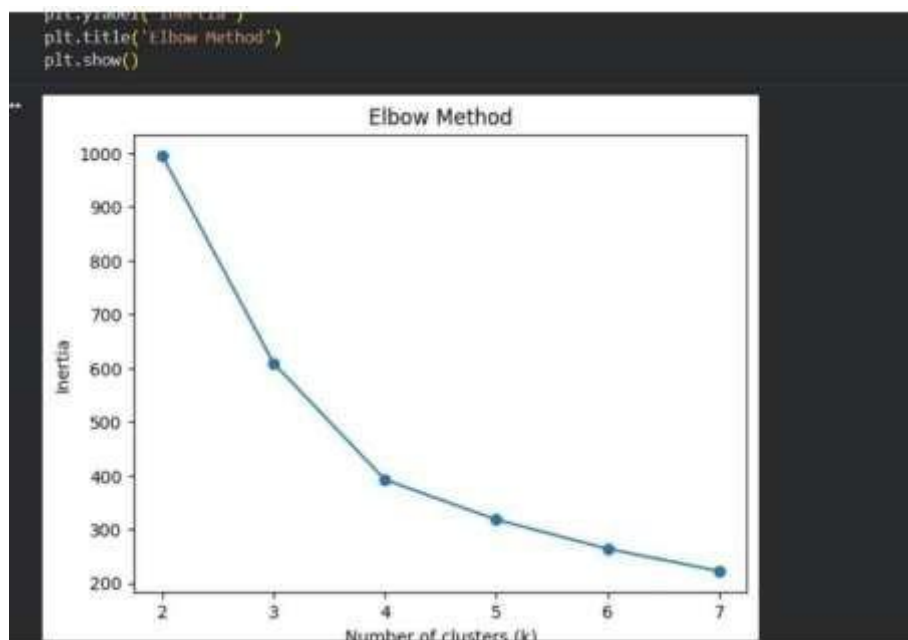
```
#3. Now the K-Means part – retry it with the fixed column:
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans

features = data[['Purchase_Frequency_Score', 'Shopping_Satisfaction']]
scaled = StandardScaler().fit_transform(features)

kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
data['cluster'] = kmeans.fit_predict(scaled)
print(data['cluster'].value_counts())
```

```
Cluster
0    316
1    250
2    234
Name: count, dtype: int64
```

Screenshot: scaling the features and running K-Means with 3 clusters



Screenshot: the Elbow Method chart used to pick 3 as the number of clusters

In short: Task 3 is about grouping customers so the business can treat different types of shoppers differently. I did it two ways — my own rule-based logic, and an unsupervised algorithm (K-Means) — and both roughly agree that customers split into 3 meaningful groups.

Task 4: Recommendation and Review Insights

This task checks whether the "trust" features on an e-commerce site — like personalized recommendations and customer reviews — actually do anything for the business. Do they really make people happier shoppers, or are they just decoration on the app?

Does a helpful recommendation = a happier customer?

I grouped customers by whether they found Snapdeal's recommendations helpful (Yes/No/Sometimes) and checked their average shopping satisfaction for each group. The result was surprisingly flat — average satisfaction barely moves between 'No' (2.89), 'Sometimes' (3.08) and 'Yes' (3.02). So recommendations being "helpful" doesn't swing satisfaction by much in this data.

```
# Task 4
#1. Recommendation helpfulness vs. shopping satisfaction

print(data.groupby('Recommendation_Helpfulness')['Shopping_Satisfaction'].mean())

Recommendation_Helpfulness
No                2.888136
Sometimes         3.077869
Yes               3.019157
Name: Shopping_Satisfaction, dtype: float64
```

Screenshot: average satisfaction grouped by recommendation helpfulness

Do reliable reviews lead to more accurate ratings?

Same idea, but this time comparing how much people trust reviews (Review_Reliability) against how accurate they think the product ratings are (Rating_Accuracy). Again the averages are all really close to each other (around 2.8 to 3.0), meaning trust in reviews isn't strongly connected to how accurate people think the ratings are.

```
#2. Review reliability & helpfulness → overall ratings

print(data.groupby('Review_Reliability')['Rating_Accuracy'].mean())
print(data.groupby('Review_Helpfulness')['Rating_Accuracy'].mean())
```

Review_Reliability	
Heavily	2.987500
Moderately	2.980645
Never	3.040541
Occasionally	2.846626
Rarely	2.971264

Name: Rating_Accuracy, dtype: float64

Review_Helpfulness	
No	2.853480
Sometimes	3.048872
Yes	2.992337

Name: Rating_Accuracy, dtype: float64

Screenshot: review reliability and review helpfulness compared against rating accuracy

Are people even paying attention to recommendations?

I also checked how often people actually engage with and trust personalized recommendations — turns out it's pretty split three ways between Yes (243), Sometimes (272), and No (285), so there's no runaway winner there either. Same story for whether people find the recommendations genuinely helpful. Basically, customers are fairly skeptical of the recommendation system as it stands — it's not winning anyone over in a big way, but it's also not driving people away.

```
#3. Trends in recommendation engagement/trust
print(data['Recommendation_Frequency_Text'].value_counts()) # use whatever you renamed it
print(data['Recommendation_Helpfulness'].value_counts())

Recommendation_Frequency_Text
No                285
Sometimes        272
Yes              243
Name: count, dtype: int64
Recommendation_Helpfulness
No                295
Yes              261
Sometimes        244
Name: count, dtype: int64

print(data.groupby('Recommendation_Helpfulness')['Shopping_Satisfaction'].mean())

Recommendation_Helpfulness
No                2.888136
Sometimes        3.077869
Yes              3.019157
Name: Shopping_Satisfaction, dtype: float64
```

Screenshot: review reliability/helpfulness compared against ratings and satisfaction

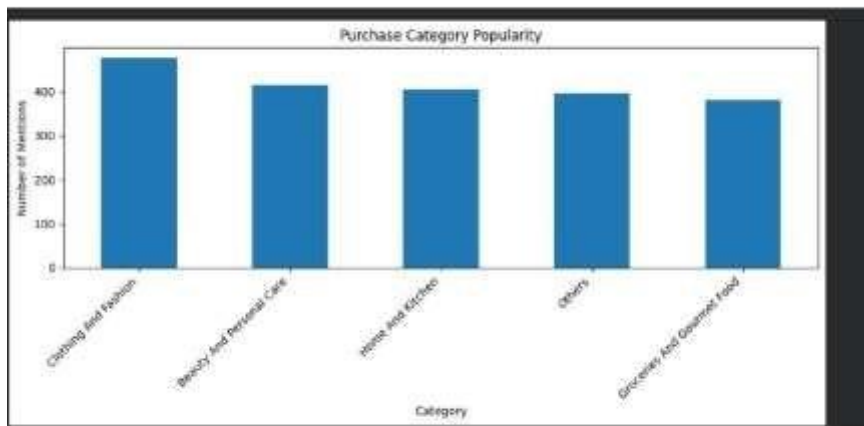
In short: Task 4 basically says — recommendations and reviews exist and people do interact with them, but in this survey they don't strongly move the satisfaction needle. My suggested takeaway for Snapdeal: the recommendation engine probably needs to get smarter/more personalized rather than just being present, because right now it isn't making a measurable difference.

Task 5: Visualization and Reporting

The last task was to pull together the most important findings from all the earlier tasks into a set of clean, presentable charts — the kind you'd actually put in front of a manager instead of a wall of numbers.

Which product categories sell the most?

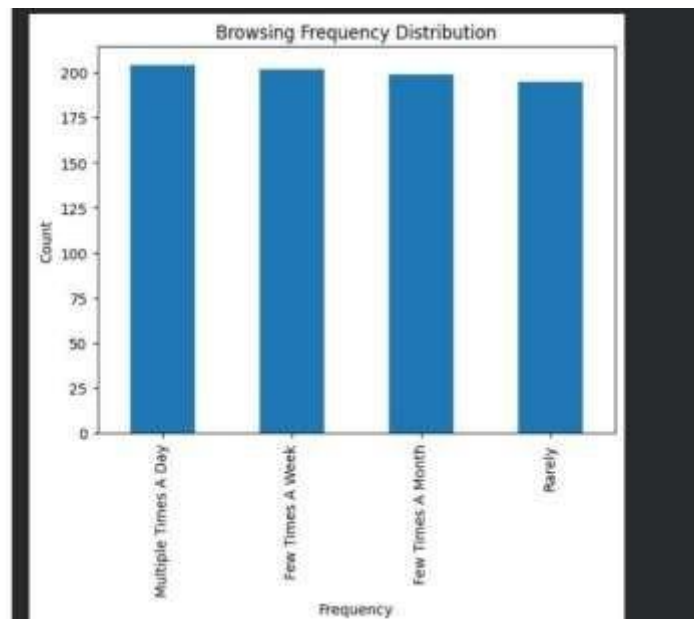
Reused the category-popularity logic from Task 2, but cleaned it up for a report — proper axis labels, rotated x-axis text so category names don't overlap, and a bigger figure size so it's actually readable when printed or presented. Clothing and Fashion again comes out clearly on top with close to 480 mentions, followed fairly closely by Beauty & Personal Care, Home & Kitchen, and Groceries.



Screenshot: final polished purchase category bar chart

Browsing habits, at a glance

A bar chart showing how often people browse Snapdeal — from multiple times a day to rarely. The counts are all pretty close to each other (195 to 205 range), so there's no single dominant browsing habit — people are spread out fairly evenly across all frequency levels.



Screenshot: browsing frequency distribution bar chart

How satisfied are people, really?

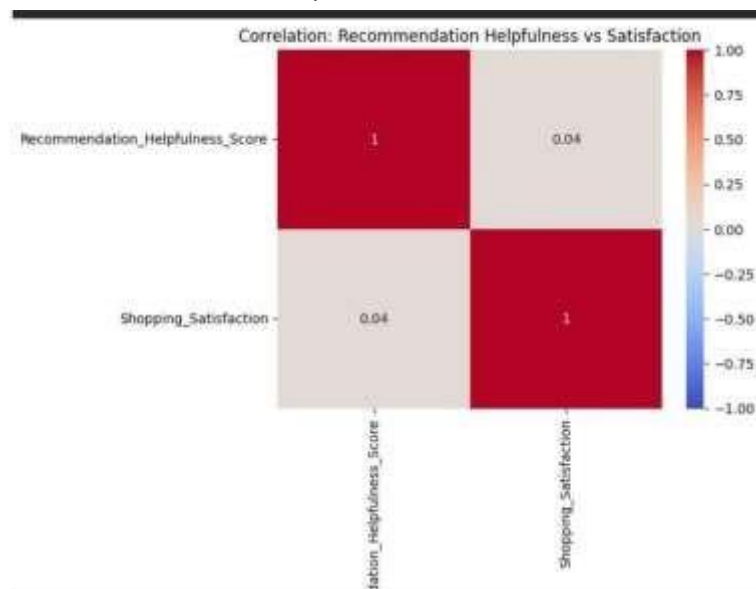
A bar chart of satisfaction scores from 1 (unhappy) to 5 (very happy). The middle score (3, i.e. neutral) is the most common answer, which usually means there's real room for Snapdeal to convert a lot of "meh" customers into genuinely happy ones with the right changes.



Screenshot: shopping satisfaction score distribution

Do recommendations and satisfaction actually correlate?

Finally, a correlation heatmap between recommendation helpfulness and shopping satisfaction. The correlation value is just 0.04 — basically zero. In plain English: there's almost no straight-line relationship between the two in this dataset, which backs up what I found back in Task 4.



Screenshot: correlation heatmap (recommendation helpfulness vs satisfaction)

In short: Task 5 packages everything into visuals — browsing habits are spread evenly, most people sit at 'neutral' satisfaction, and the recommendation system has basically no measurable correlation with how happy people are. These charts are the ones I'd actually show in a presentation.

Overall takeaway: I cleaned messy survey data, explored who the customers are and how they shop, split them into meaningful segments using both simple rules and K-Means clustering, checked whether

recommendations/reviews actually help (they barely do, based on this data), and wrapped it all up in charts that tell the story at a glance.

Video Recording

<https://www.loom.com/share/10ab7cea938741809ebc0fb4cc253576>